Here, we've used with(...) to constrain which parameter values this stub applies to. We'll talk more about with later in *Constraining Arguments*, on page 256.

Modifying the Return Value

Sometimes, you want to slightly change the behavior of the method you're stubbing, rather than replacing it outright. You may, for instance, need to modify its return value.

To do so, call RSpec's and_wrap_original method, passing it a block containing your custom behavior. Your block will take the original implementation as an argument, which you can call at any time.

Here, we use this technique to stub out a CustomerService API to return a subset of customers:

14-customizing-test-doubles/01/configuring_responses.rb

```
allow(CustomerService).to receive(:all).and_wrap_original do |original|
  all_customers = original.call
  all_customers.sort_by(&:id).take(10)
end
```

This technique can be handy for acceptance specs, where you want to test against a live service. If the vendor doesn't provide a test API that only returns a few records, you can call the real API and narrow down the records yourself. By working on just a subset of the data, your specs will remain snappy.

Tweaking Arguments

You can also use and_wrap_original to tweak the arguments you pass *into* a method. This technique comes in handy when the code you're testing uses a lot of hard-coded values.

In *Stubbed Constants*, on page 244, we used stub_const to call a hashing algorithm with a lower cost factor so that our specs kept running quickly. That approach only worked because the cost was defined as a constant.

If instead the number had been a hard-coded argument value, we could have overridden it using and_wrap_original:

14-customizing-test-doubles/01/configuring_responses.rb

```
allow(PasswordHash).to receive(:hash_password)
  .and_wrap_original do |original, cost_factor|
    original.call(1)
  end
```